



DevOps Evaluation — Interview Preparation Guide

*CloudMart Enterprise: Cloud-Native E-Commerce
Observability Platform on Google Cloud*

*CTS Hackathon 2026 | GCP Project
Prepared for: DevOps Team Evaluation*

Team Members	Responsibility
Varunshiyam	Lead — Architecture, Gateway, Looker Pipeline, Logging
Babu Senthil	Cloud Run Deployments
Akash	GKE (Kubernetes) Deployments
Roshni & Swathi	Compute Engine (GCE) Deployments
Ecclesiastes & Emayan	BigQuery Analytics & Views

Table of Contents

1. Project Overview & Architecture
2. GCP Services Used — Quick Reference
3. Possible DevOps Evaluation Questions & Answers
 - 3.1 Architecture & Microservices
 - 3.2 Containerization & Docker
 - 3.3 Google Kubernetes Engine (GKE)
 - 3.4 Cloud Run (Serverless)
 - 3.5 Compute Engine (GCE / VMs)
 - 3.6 Cloud SQL & Database
 - 3.7 Networking & Service Discovery
 - 3.8 Observability — Logging, Monitoring & Alerting
 - 3.9 BigQuery & Analytics Pipeline
 - 3.10 Looker Studio & Dashboards
 - 3.11 CI/CD & Deployment Strategy
 - 3.12 Security & IAM
 - 3.13 Cost Optimization
 - 3.14 Pub/Sub & Event-Driven Architecture
 - 3.15 Troubleshooting & Incident Response
4. Conceptual Topics to Master
5. Quick-Revision Cheat Sheet

1. Project Overview & Architecture

SYNTRIX (CloudMart) is a production-grade, cloud-native e-commerce platform designed to generate rich structured JSON telemetry. It serves as a realistic workload for **GCP observability demonstrations**, showcasing how to monitor, log, alert on, and visualize a distributed microservices system deployed across **three GCP compute platforms**: Cloud Run, GKE, and Compute Engine.

Architecture Diagram

```
Customer → CloudMart Web Portal → API Gateway (Cloud Run, Port 8000)
  ■■ Catalog Service (Cloud Run)
  ■■ Cart Service (Cloud Run)
  ■■ Order Service (GKE) → Payment Service (GKE)
  ■■ Inventory Service (Compute Engine / GCE)
      ■■ Cloud SQL (PostgreSQL 15)
```

Key Design Decisions

- **Multi-Compute Strategy**: Stateless services on Cloud Run (scale-to-zero), orchestration-heavy services on GKE, stateful/legacy on GCE — mirrors real enterprise patterns.
- **Structured JSON Logging**: All services emit native JSON logs to stdout → ingested by Cloud Logging without grok parsing.
- **Distributed Tracing**: X-Request-Id and X-Trace-Id propagated via Python ContextVars across all service boundaries.
- **Observability Pipeline**: Cloud Logging → GCS → BigQuery → 6 Analytical Views → Looker Studio Dashboards.
- **Simulated Failures**: Developer Panel toggles (DB delays, payment timeouts, stock exhaustion) produce organic error logs.
- **Event-Driven**: Order events published to Google Cloud Pub/Sub for downstream consumers.

2. GCP Services Used — Quick Reference

GCP Service	Used For	Key Concepts
Cloud Run	Gateway, Catalog, Cart services	Serverless containers, scale-to-zero, revision-based deployment
GKE Autopilot	Order & Payment services	Kubernetes, Deployments, Services, Secrets, Internal LB
Compute Engine	Inventory service	VM instances, startup scripts, firewall rules, metadata server
Cloud SQL	PostgreSQL 15 database	Managed DB, instance tiers, IAM, Cloud SQL Proxy
Artifact Registry	Docker image storage	Container registry, regional repositories, image tagging
Cloud Logging	Centralized log collection	Structured logs, Log Router, log sinks, jsonPayload
Cloud Monitoring	Alert policies	Metric-based alerts, notification channels, thresholds
BigQuery	Log analytics & views	NDJSON loading, SQL views, APPROX_QUANTILES, SAFE_DIVIDE
Looker Studio	Dashboard visualization	6-sheet dashboard, KPI scorecards, cross-filters
Cloud Storage (GCS)	Log staging bucket	gsutil, NDJSON upload, bucket lifecycle
Secret Manager	Database URL storage	Secret versions, IAM bindings, service account access
Cloud Pub/Sub	Order event streaming	Topics, publish/subscribe, event-driven architecture
IAM	Service account permissions	Roles, policy bindings, least-privilege principle
VPC / Firewall	Network security	Firewall rules, internal traffic, source ranges

3. Possible DevOps Evaluation Questions & Answers

Below are categorized questions a DevOps evaluation panel may ask about this project. Each question is followed by a model answer referencing your actual codebase.

3.1 Architecture & Microservices

Q: Explain the overall architecture of SYNTRIX. Why did you choose a microservices pattern?

Answer: SYNTRIX (CloudMart) is a cloud-native e-commerce platform with 5 independent backend microservices (Catalog, Cart, Order, Payment, Inventory) fronted by an API Gateway. We chose microservices because: (1) Each service can scale independently — e.g., Catalog handles high read traffic on Cloud Run while Order handles transactional orchestration on GKE. (2) Independent deployability — we can update Payment without redeploying Catalog. (3) Technology diversity — each service owns its domain logic. (4) Fault isolation — a failing Payment service doesn't crash the entire system.

Q: How does inter-service communication work?

Answer: Synchronous HTTP/REST via the API Gateway pattern. The Gateway (gateway/main.py) uses `httpx.AsyncClient` to proxy requests to downstream services. Service URLs are configured via environment variables (`CATALOG_URL`, `CART_URL`, `ORDER_URL`, etc.). In the cloud environment, Cloud Run identity tokens are fetched from the GCE metadata server for service-to-service authentication. All requests propagate `X-Request-Id` and `X-Trace-Id` headers for distributed tracing.

Q: Why did you deploy different services on different GCP compute platforms?

Answer: This was a deliberate architectural decision to demonstrate GCP's multi-compute ecosystem: (1) Cloud Run for stateless, fast-scaling services (Gateway, Catalog, Cart) — leverages scale-to-zero for cost efficiency. (2) GKE Autopilot for complex orchestration (Order, Payment) — Order service coordinates with Cart, Inventory, and Payment, requiring stable networking and Kubernetes secrets management. (3) Compute Engine for Inventory — simulates a stateful/legacy service running on a VM, demonstrating GCE startup scripts and metadata-driven configuration.

Q: What is the role of the API Gateway in your architecture?

Answer: The API Gateway (gateway/main.py, deployed on Cloud Run) serves as: (1) Reverse Proxy — routes `/api/{service}/*` to the correct backend service dynamically using a SERVICES map. (2) Authentication Layer — in cloud mode, it fetches identity tokens from the metadata server for downstream calls. (3) Trace Propagation — generates and forwards `X-Request-Id` and `X-Trace-Id`. (4) Access Logging — logs every request with endpoint, method, status_code, latency_ms. (5) Static File Server — serves the frontend SPA via FastAPI's StaticFiles mount.

3.2 Containerization & Docker

Q: Explain your Dockerfile. Why did you use a single Dockerfile for all services?

Answer: We use a single multi-purpose Dockerfile with a BUILD ARG called SERVICE_NAME. This allows us to build any service from the same Dockerfile by passing --build-arg SERVICE_NAME=services/order. The Dockerfile: (1) Uses python:3.12-slim as the base for minimal image size. (2) Installs dependencies from requirements.txt. (3) Copies the shared/ library (logging, middleware, models). (4) Copies only the specific service code using COPY \${SERVICE_DIR}. (5) Uses a dynamic CMD that constructs the uvicorn module path from SERVICE_DIR. This reduces Dockerfile duplication across 6 services.

Q: Explain docker-compose.yml. What is the x-service-common YAML anchor?

Answer: docker-compose.yml orchestrates the entire system locally. The x-service-common YAML anchor (&service-common;) defines shared configuration: same Dockerfile, DATABASE_URL env var, and depends_on postgres with healthcheck. Each service then uses '<<: *service-common' to inherit this config and override specific build args and env vars. The postgres service uses pg_isready healthcheck. The 'seed' service runs once (service_completed_successfully) to populate initial data before other services start.

Q: Why do you use docker buildx with --platform linux/amd64?

Answer: We develop on Apple Silicon (ARM64) Macs, but GCP services (Cloud Run, GKE, GCE) run on AMD64 (x86_64) architecture. docker buildx build --platform linux/amd64 performs cross-compilation to ensure the images are compatible with GCP's infrastructure. Without this, ARM-built images would fail to run on GCP.

Q: How do you handle image versioning and tagging?

Answer: Images are tagged with semantic versions (e.g., order:1.0.2, gateway:1.0.0) and pushed to GCP Artifact Registry at asia-south1-docker.pkg.dev/{PROJECT_ID}/syntrix-cloudmart/{service}:{version}. The registry is regional (asia-south1) to minimize pull latency from the deployment region.

3.3 Google Kubernetes Engine (GKE)

Q: Why did you use GKE Autopilot instead of Standard mode?

Answer: GKE Autopilot (gcloud container clusters create-auto) was chosen because: (1) No node management — Google manages the nodes, OS patches, and scaling. (2) Pod-level billing — we only pay for pods running, not idle nodes. (3) Built-in best practices — security hardening, workload identity, and resource optimization are automatic. (4) Faster setup for hackathon — 5-10 minutes to provision vs. manual node pool configuration.

Q: Explain the Kubernetes manifests in deploy_gke.sh.

Answer: We define two Deployments (order-deployment, payment-deployment) and two Services. Each Deployment specifies: replicas=1, container image from Artifact Registry, containerPort=8080, and DATABASE_URL from a Kubernetes Secret (db-secret). The Services use type=LoadBalancer with annotation networking.gke.io/load-balancer-type=Internal for private internal load balancing — services are only accessible within the VPC, not from the public internet. Port mapping: external port 8000 →

container port 8080.

Q: How do you manage secrets in GKE?

Answer: Database credentials are stored in a Kubernetes Secret created via: `kubectl create secret generic db-secret --from-literal=DATABASE_URL=... --dry-run=client -o yaml | kubectl apply -f -`. The `--dry-run=client | apply` pattern makes the command idempotent. Pods reference the secret via `env[].valueFrom.secretKeyRef`. Image pull secrets (`gcr-json-key`) are also configured for Artifact Registry access.

Q: What is an Internal Load Balancer and why did you use it?

Answer: An Internal Load Balancer (ILB) makes a Kubernetes Service accessible only within the GCP VPC — not from the internet. We use annotation `networking.gke.io/load-balancer-type=Internal` because Order and Payment services should only be called by the API Gateway (via service-to-service communication), never by external users directly. This follows the principle of least exposure.

3.4 Cloud Run (Serverless)

Q: How did you deploy services to Cloud Run?

Answer: Using `gcloud run deploy` with: (1) `--image` pointing to Artifact Registry, (2) `--allow-unauthenticated` for public access (Gateway/Catalog), (3) `--set-env-vars` for runtime configuration, (4) `--region=asia-south1` for data residency. IAM permissions (`roles/run.invoker` for allUsers) are applied separately. The deployment returns the service URL which is used to configure inter-service communication.

Q: What is Cloud Run's scale-to-zero capability and why is it important?

Answer: Cloud Run automatically scales instances down to zero when no requests are received. When a request arrives, it cold-starts a new instance in ~1-2 seconds. This is cost-efficient because: (1) You only pay for actual compute time (per-request billing). (2) Low-traffic services like Catalog don't incur charges during idle periods. (3) It auto-scales up during traffic spikes without manual intervention.

Q: How does service-to-service authentication work on Cloud Run?

Answer: In our `gateway/main.py`, when `ENVIRONMENT=cloud`, the Gateway fetches an identity token from the GCE metadata server (`http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/identity`) with the downstream service URL as the audience. This OIDC token is then sent as a Bearer token in the Authorization header. The downstream Cloud Run service validates this token automatically.

3.5 Compute Engine (GCE / VMs)

Q: How is the Inventory service deployed on Compute Engine?

Answer: `deploy_inventory.sh`: (1) Builds and pushes the Docker image to Artifact Registry. (2) Creates a VM (`e2-micro`, `asia-south1-a`) with a startup script. (3) The startup script installs Docker, pulls the image, and runs the container with `DATABASE_URL` from instance metadata. (4) DB IP is fetched from Cloud

SQL via gcloud sql instances describe. (5) A firewall rule (allow-inventory-internal) allows TCP:8080 from internal VPC (10.0.0.0/8) only.

Q: What is the GCE metadata server and how do you use it?

Answer: The GCE metadata server (<http://metadata.google.internal>) provides instance-level configuration. In our setup, the VM's startup script reads the database-url from custom metadata using: `curl -s 'http://metadata.google.internal/computeMetadata/v1/instance/attributes/database-url' -H 'Metadata-Flavor: Google'`. This avoids hardcoding credentials in the startup script.

Q: What machine type did you use and why?

Answer: e2-micro — the smallest general-purpose machine type in GCE. It provides 0.25 vCPU (burstable to 2 vCPUs) and 1 GB RAM. Sufficient for the Inventory service's workload. Chosen for cost efficiency in a hackathon context. In production, you'd use e2-small or e2-medium based on load testing results.

3.6 Cloud SQL & Database

Q: How did you set up Cloud SQL?

Answer: setup_cloudsql.sh: (1) Creates a Cloud SQL PostgreSQL 15 instance (db-f1-micro tier) in asia-south1. (2) Creates the 'cloudmart' database and 'cloudmart_user' with a random password (openssl rand -hex 12). (3) Stores the connection string in Secret Manager with automatic replication. (4) Creates a dedicated service account (cloudrun-app-sa) with roles/cloudsql.client and roles/secretmanager.secretAccessor — following least privilege.

Q: What is the Cloud SQL Proxy and why would you use it?

Answer: The Cloud SQL Proxy creates a secure, encrypted tunnel between your application and Cloud SQL without requiring VPC whitelisting or SSL certificate management. The connection string format postgresql://user:pass@/db?host=/cloudsql/{connection_name} uses Unix socket via the proxy. However, in our GKE/GCE deployments, we connect directly to the SQL instance's public IP for simplicity.

Q: What ORM do you use and how is the database schema managed?

Answer: We use SQLAlchemy as the ORM with Alembic for migrations. Models are defined in shared/models/models.py. The seed service (database/seed_data/seed.py) creates tables and inserts 50 mock products on first startup. Alembic configuration is in database/alembic.ini for schema version tracking.

3.7 Networking & Service Discovery

Q: How do services discover each other?

Answer: Environment variable-based service discovery. Each service's URL is injected at deployment time: (1) Docker Compose: http://catalog:8000 (Docker DNS), (2) Cloud Run: full HTTPS URL from deployment output, (3) GKE: Kubernetes Service DNS (order-service.default.svc.cluster.local via ILB),

(4) GCE: Direct IP with port. The Gateway reads CATALOG_URL, CART_URL, etc. from env vars.

Q: What firewall rules did you configure?

Answer: allow-inventory-internal: Allows TCP:8080 from 10.0.0.0/8 (all internal VPC IPs) to VMs tagged with 'inventory-server'. This ensures only internal services (Gateway, Order) can reach the Inventory service on GCE, not external traffic. Cloud Run services are publicly accessible via IAM (allUsers: roles/run.invoker).

3.8 Observability — Logging, Monitoring & Alerting

Q: Explain your structured logging implementation.

Answer: shared/logging/logger.py defines a CustomJsonFormatter extending python-json-logger. Every log entry automatically includes: timestamp (ISO 8601), severity, service name, environment, request_id (from ContextVar), trace_id (from ContextVar), and Cloud Logging trace format (logging.googleapis.com/trace). Logs are written to stdout as JSON — Cloud Logging's agent automatically ingests them from Cloud Run, GKE, and GCE without any custom log shipping.

Q: How does distributed tracing work in your system?

Answer: shared/middleware/tracing.py implements TracingMiddleware (Starlette BaseHTTPMiddleware). On each request: (1) Extracts or generates X-Request-Id and X-Trace-Id from headers. (2) Stores them in Python ContextVars (thread-safe async context). (3) When the Gateway proxies to downstream services, it forwards these headers. (4) Each downstream service's TracingMiddleware picks them up — creating a distributed trace across all 5 services. The logger injects trace_id into every log entry for end-to-end correlation.

Q: Describe your alerting strategy.

Answer: create_alerts.sh configures 5 Cloud Monitoring alert policies: (1) Cloud Run 5xx Errors — threshold >10 in 60s → notifies Babu. (2) GKE High CPU — CPU utilization >80% for 180s → notifies Akash. (3) GCE High CPU — CPU utilization >85% for 180s → notifies Roshni & Swathi. (4) BigQuery High Execution Time — query >10s → notifies Ecclesiastes & Emayan. (5) Log Pipeline Errors — global ERROR log count >5 in 60s → notifies Varunshiyam. Each alert uses email notification channels created via gcloud beta monitoring channels create.

Q: What is the difference between Cloud Logging and Cloud Monitoring?

Answer: Cloud Logging collects, stores, and searches log data (text/JSON entries). Cloud Monitoring collects metrics (CPU, memory, request counts, custom metrics) and creates alerts. They work together: metrics trigger alerts, and you investigate the root cause by searching logs. In SYNTRIX, Cloud Logging ingests our JSON logs, while Cloud Monitoring watches system metrics (run.googleapis.com/request_count, kubernetes.io/container/cpu/limit_utilization) for alerting.

Q: How do your logs go from application to Looker Studio dashboard?

Answer: The full pipeline: (1) Services emit JSON to stdout → Cloud Logging auto-ingests. (2) export_logs.sh runs 'gcloud logging read' to extract logs as JSON. (3) prepare_bq_json.py converts to NDJSON format with sanitized keys. (4) gsutil uploads to GCS bucket. (5) bq load imports into BigQuery

(syntrix_logs.raw_logs). (6) Additionally, realtime_log_streamer.py polls Cloud Logging every 10 seconds for near-real-time streaming. (7) 6 BigQuery SQL views aggregate the raw data. (8) Looker Studio connects to these views.

3.9 BigQuery & Analytics Pipeline

Q: Explain your BigQuery views and their purpose.

Answer: 6 views in syntrix_logs dataset: (1) `looker_cleaned_logs` — base view normalizing resource types, extracting jsonPayload fields. (2) `looker_latency_analysis` — hourly AVG/MAX latency per service/endpoint. (3) `looker_error_analysis` — hourly 4xx/5xx counts with error_rate_percentage. (4) `looker_usage_monitoring` — daily request volume per environment/endpoint. (5) `looker_performance_kpis` — P50/P90/P99 latency percentiles using APPROX_QUANTILES. (6) `looker_cost_optimization_recommendations` — rules-based cost-saving suggestions based on 24h traffic.

Q: What is APPROX_QUANTILES in BigQuery and why use it?

Answer: `APPROX_QUANTILES(latency_ms, 100)` returns an array of 101 values representing percentile boundaries. `OFFSET(50)` gives P50 (median), `OFFSET(90)` gives P90, `OFFSET(99)` gives P99. It's an approximate function that's much faster than exact quantile computation on large datasets. P50/P90/P99 latencies are critical SRE metrics — P99 shows worst-case user experience.

Q: What is SAFE_DIVIDE and why use it instead of regular division?

Answer: `SAFE_DIVIDE(a, b)` returns NULL instead of throwing a division-by-zero error when `b=0`. In our error analysis view, some service/time combinations may have zero total requests. Regular division would fail; `SAFE_DIVIDE` gracefully handles edge cases.

Q: How does your cost optimization view work?

Answer: `looker_cost_optimization_recommendations` uses a CASE statement on 24h request counts: - Low traffic (<100 reqs) on GKE → 'Scale down GKE nodes or migrate to Cloud Run'. - Low traffic (<100) on GCE → 'Downgrade VM instance type or migrate to Cloud Run'. - High traffic (>10000) on Cloud Run → 'Consider migrating to GKE for sustained discounts'. - Otherwise → 'Traffic Optimized for Environment'. Uses CROSS JOIN with a CTE to get the latest timestamp.

3.10 Looker Studio & Dashboards

Q: Describe your dashboard architecture.

Answer: 6-sheet multi-layer dashboard: (1) Executive Overview — KPI scorecards, traffic trends, error rates. (2) System Performance — P50/P90/P99 latency, latency over time by service. (3) Error Analysis — 4xx/5xx breakdown, error rate trends, top error messages. (4) Infrastructure Usage — requests by compute environment, treemap by endpoint. (5) Cost Optimization — recommendation table with conditional formatting. (6) Developer Log Explorer — raw log table with severity/status filters. All sheets share a left navigation panel and global date/service/environment filters.

Q: How do global filters work in Looker Studio?

Answer: Date range controls and drop-down filters placed on each sheet automatically cross-filter all charts that share the same data source. For charts using different data sources, we set up data source filter binding in chart properties. This ensures selecting 'order-service' in the filter updates all charts across latency, error, and usage panels simultaneously.

3.11 CI/CD & Deployment Strategy

Q: Describe your deployment pipeline.

Answer: Our deployment is scriptified in multiple shell scripts: (1) `setup_cloudsql.sh` — provisions Cloud SQL, secrets, service accounts. (2) `deploy_phase1.sh` — builds and deploys Gateway + Cart to Cloud Run. (3) `setup_gke_cluster.sh` — creates GKE Autopilot cluster. (4) `deploy_gke.sh` — builds, pushes, and deploys Order + Payment to GKE. (5) `deploy_inventory.sh` — deploys Inventory on Compute Engine VM. (6) `create_alerts.sh` — configures monitoring alerts. (7) `export_logs.sh` — extracts logs to BigQuery. In production, these would be CI/CD pipeline stages in Cloud Build or GitHub Actions.

Q: How would you improve this deployment for production?

Answer: (1) Use Cloud Build or GitHub Actions for automated CI/CD. (2) Implement Terraform/Pulumi for Infrastructure as Code (IaC). (3) Add automated testing (unit, integration, e2e) in the pipeline. (4) Use Cloud Deploy for managed deployment with canary/blue-green strategies. (5) Implement GitOps with ArgoCD for Kubernetes deployments. (6) Add image vulnerability scanning with Container Analysis.

3.12 Security & IAM

Q: How do you handle secrets and credentials?

Answer: (1) Cloud SQL password stored in Secret Manager with IAM-bound access. (2) GKE uses Kubernetes Secrets for `DATABASE_URL`. (3) GCE uses instance metadata for the database URL. (4) Service accounts follow least-privilege: `cloudrun-app-sa` has only `cloudsql.client` and `secretmanager.secretAccessor` roles. (5) In a real scenario, we'd use Workload Identity for GKE instead of static secrets.

Q: What security improvements would you make?

Answer: (1) Enable VPC Service Controls for perimeter-based security. (2) Use Workload Identity to bind Kubernetes SAs to GCP SAs. (3) Remove `--allow-unauthenticated` on Cloud Run; use IAP instead. (4) Implement Cloud Armor WAF for DDoS protection. (5) Enable Binary Authorization for image verification. (6) Use private Cloud SQL connections via Private Service Access. (7) Rotate secrets automatically with Secret Manager's rotation feature.

3.13 Cost Optimization

Q: How does your project address cost optimization?

Answer: (1) Cloud Run's scale-to-zero for low-traffic services (no idle cost). (2) GKE Autopilot's pod-level billing (no idle node cost). (3) GCE e2-micro for minimal VM cost. (4) Cloud SQL db-f1-micro tier for hackathon. (5) BigQuery view `looker_cost_optimization_recommendations` analyzes traffic patterns and suggests migration actions (e.g., move low-traffic GKE services to Cloud Run). (6) Regional deployment in `asia-south1` to avoid inter-region data transfer costs.

Q: What are Committed Use Discounts (CUDs) and Sustained Use Discounts (SUDs)?

Answer: CUDs: 1-3 year commitments for a fixed amount of vCPUs/memory at 57-70% discount. SUDs: Automatic discounts for running GCE/GKE instances >25% of the month (up to 30% off). Our cost optimization view suggests migrating high-traffic Cloud Run services to GKE to take advantage of SUDs/CUDs for sustained workloads.

3.14 Pub/Sub & Event-Driven Architecture

Q: How do you use Google Cloud Pub/Sub?

Answer: In `services/order/main.py`, when an order is created, we publish an `'order.created'` event to the `'order-events'` Pub/Sub topic. The event payload contains `order_id`, `customer_id`, `total_amount`, `items`, and `trace_id`. This enables downstream consumers (analytics, notifications, fulfillment) to react to order events asynchronously without coupling to the Order service.

Q: What is the difference between Pub/Sub and synchronous HTTP calls?

Answer: Pub/Sub is asynchronous — the publisher doesn't wait for consumers. Benefits: (1) Decoupling — publisher doesn't know about subscribers. (2) Buffering — messages are retained if subscribers are temporarily offline. (3) Fan-out — multiple subscribers can process the same event. (4) Reliability — at-least-once delivery guarantee. HTTP calls are synchronous — the caller blocks until the response. Used for real-time operations like inventory reservation where the result is needed immediately.

3.15 Troubleshooting & Incident Response

Q: A user reports slow checkout. How would you debug it?

Answer: (1) Check Looker Dashboard Sheet 2 (Performance) for P99 latency spikes on order-service. (2) In Cloud Logging, search for the user's X-Trace-Id to see the full request flow. (3) Check individual service latencies: `gateway` → `cart` → `inventory` → `payment` to find the bottleneck. (4) Check Cloud Monitoring alerts for CPU/memory thresholds being breached. (5) In BigQuery, query `looker_latency_analysis` for the `/checkout` endpoint's recent avg/max latency. (6) If database-related, check Cloud SQL metrics (connections, CPU, query latency).

Q: How does the Developer Panel help in troubleshooting?

Answer: The Dev Panel (`frontend/admin.js`) allows toggling simulated failures: (1) Database Delay — injects artificial latency into DB operations. (2) Payment Timeout — causes payment service to fail. (3) Out of Stock — simulates inventory exhaustion. These generate organic error logs (ERROR severity, specific error messages) that flow through the entire observability pipeline, allowing us to test alerting, logging, and dashboard response.

Q: 5xx errors spike on Cloud Run. What's your incident response?

Answer: (1) Acknowledge the Cloud Monitoring alert (email notification to Babu). (2) Check Cloud Run revision logs in Cloud Logging for error details. (3) Check if it's a cold-start issue (scale-from-zero latency) or application error. (4) If application error — check recent deployments, roll back the revision if needed. (5) If infrastructure — check Cloud SQL connectivity, downstream service health. (6) Post-incident: update the BigQuery dashboard, document in a post-mortem.

4. Conceptual Topics to Master

These are the core DevOps/Cloud concepts demonstrated in SYNTRIX that you should deeply understand.

4.1 Microservices Architecture

A design pattern where an application is composed of small, independent, loosely-coupled services. Each service owns its own data, runs in its own process, and communicates via lightweight protocols (HTTP/REST, gRPC).

SYNTRIX Example: 5 independent services (Catalog, Cart, Order, Payment, Inventory) each with its own FastAPI app.

Key Concepts: Service decomposition, bounded contexts (DDD), independent deployability, polyglot persistence, API Gateway pattern, service mesh, saga pattern (Order orchestrates Cart→Inventory→Payment).

4.2 Containerization & Docker

Packaging an application with all its dependencies into a portable container image.

Key Concepts: Docker images vs containers, Dockerfile instructions (FROM, COPY, RUN, CMD, ARG, ENV), multi-stage builds, layer caching, .dockerignore, image registries (Artifact Registry), docker-compose for local orchestration, YAML anchors (&/*), container networking (bridge, host), volume mounts, healthchecks, docker buildx for cross-architecture builds.

4.3 Kubernetes (K8s) Fundamentals

Container orchestration platform for automating deployment, scaling, and management.

Key Concepts: Pods, Deployments, Services (ClusterIP, NodePort, LoadBalancer), Secrets, ConfigMaps, namespaces, labels/selectors, rolling updates, ReplicaSets, kubectl CLI, YAML manifests, Horizontal Pod Autoscaler (HPA), liveness/readiness probes, resource requests/limits.

GKE Autopilot vs Standard: Autopilot manages nodes automatically with pod-level billing; Standard gives full node control but requires more operational overhead.

4.4 Serverless Computing (Cloud Run)

Fully managed compute platform that automatically scales containerized applications.

Key Concepts: Scale-to-zero, cold starts, concurrency settings, revision-based deployments, traffic splitting (canary deployments), VPC connectors, IAM-based access control, per-request billing, maximum instances limit, startup CPU boost.

When to use: Stateless HTTP services, event-driven processing, APIs with variable traffic.

4.5 Virtual Machines & Compute Engine

IaaS offering providing customizable virtual machines on GCP.

Key Concepts: Machine types (e2, n2, c2), preemptible/spot VMs, startup scripts, instance metadata server, SSH access, instance groups (managed/unmanaged), firewall rules (ingress/egress, tags, source ranges), load balancing, snapshots/images.

When to use: Stateful applications, legacy workloads, GPU/TPU workloads, specific OS needs.

4.6 Cloud SQL (Managed Database)

Fully managed relational database service supporting PostgreSQL, MySQL, SQL Server.

Key Concepts: Instance tiers, high availability (regional), read replicas, automated backups, point-in-time recovery, maintenance windows, Cloud SQL Proxy (secure tunneling), Private IP vs Public IP, connection limits, performance insights.

SYNTRIX: PostgreSQL 15, db-f1-micro tier, public IP with authorized networks.

4.7 Structured Logging & Observability

The practice of emitting logs in a machine-parseable format (JSON) with consistent fields.

Key Concepts: The three pillars of observability: Logs, Metrics, Traces. Structured vs unstructured logging, log levels (DEBUG, INFO, WARNING, ERROR, CRITICAL), log aggregation (Cloud Logging), log sinks, log-based metrics, jsonPayload field in Cloud Logging, correlation IDs for distributed tracing, python-json-logger library.

4.8 Distributed Tracing

Tracking a request as it flows through multiple microservices.

Key Concepts: Trace ID (spans entire transaction), Span ID (individual service hop), context propagation (W3C Trace Context, B3 headers), ContextVars (Python async-safe context), Cloud Trace integration (logging.googleapis.com/trace format), OpenTelemetry (industry standard).

SYNTRIX: X-Request-Id and X-Trace-Id headers propagated via ContextVars.

4.9 Cloud Monitoring & Alerting

Monitoring system health and automatically notifying teams of anomalies.

Key Concepts: Metrics (system vs custom), alert policies (condition, threshold, duration), notification channels (email, SMS, PagerDuty, Slack), uptime checks, SLIs/SLOs/SLAs, dashboards vs alerts, incident management, alert fatigue, conditional threshold.

SYNTRIX: 5 alert policies covering Cloud Run, GKE, GCE, BigQuery, and global errors.

4.10 BigQuery & Data Analytics

Serverless, highly-scalable data warehouse for analytics.

Key Concepts: Datasets, tables, views, NDJSON format, schema auto-detection, partitioning (by timestamp for cost/performance), clustering, SQL dialects, APPROX_QUANTILES (approximate percentiles), SAFE_DIVIDE (null-safe division), TIMESTAMP_TRUNC (time bucketing), JSON_EXTRACT_SCALAR (parsing nested JSON), materialized views, scheduled queries, data freshness.

4.11 Looker Studio (Data Visualization)

Free Google tool for creating interactive dashboards connected to various data sources.

Key Concepts: Data sources, calculated fields, scorecards, time series charts, bar charts, donut charts, treemaps, tables with heatmaps, pivot tables, report-level vs page-level vs chart-level filters, filter controls, blended data, drill-down, conditional formatting, themes, multi-page reports with navigation.

4.12 Infrastructure as Code (IaC)

Managing infrastructure through declarative configuration files rather than manual setup.

Key Concepts: Terraform, Pulumi, Cloud Deployment Manager, declarative vs imperative, state management, plan/apply workflow, modules, remote state, drift detection.

SYNTRIX Status: Uses shell scripts (imperative); production would use Terraform (declarative).

4.13 CI/CD Pipelines

Continuous Integration (automated builds/tests) and Continuous Deployment (automated releases).

Key Concepts: Cloud Build, GitHub Actions, build triggers, build steps, artifact storage, deployment strategies (rolling, canary, blue-green), testing stages (unit, integration, e2e), environment promotion (dev → staging → prod), rollback mechanisms, feature flags, GitOps.

4.14 Networking Fundamentals

Understanding how services communicate in a cloud environment.

Key Concepts: VPC (Virtual Private Cloud), subnets, IP ranges (CIDR notation), firewall rules (ingress/egress), load balancers (internal vs external), DNS resolution, ports/protocols, NAT, VPN/Interconnect, Private Google Access, service discovery (DNS-based, env var-based, service mesh).

SYNTRIX: Internal LB for GKE, firewall rules for GCE, env-var service discovery.

4.15 IAM & Security

Identity and Access Management — controlling who can do what on which resources.

Key Concepts: Principals (users, service accounts, groups), roles (primitive, predefined, custom), policy bindings, least privilege principle, service accounts, Workload Identity, Secret Manager, encryption at rest/in transit, Cloud Armor, Binary Authorization, VPC Service Controls, audit logging.

SYNTRIX: Dedicated SA (cloudrun-app-sa) with cloudsql.client and secretmanager.secretAccessor.

4.16 Event-Driven Architecture & Pub/Sub

Asynchronous communication pattern where services produce and consume events.

Key Concepts: Publishers, subscribers, topics, subscriptions, at-least-once delivery, dead-letter topics, message ordering, message retention, push vs pull subscriptions, fan-out pattern, event sourcing, CQRS.

SYNTRIX: Order service publishes 'order.created' events to Pub/Sub topic.

4.17 API Gateway Pattern

A single entry point for all client requests that routes to appropriate backend services.

Key Concepts: Reverse proxy, request routing, rate limiting, authentication/authorization, request/response transformation, circuit breaker, retry logic, load balancing, API versioning, CORS handling.

SYNTRIX: Custom Gateway using FastAPI with dynamic routing based on URL path segments.

4.18 SRE Concepts — SLIs, SLOs, SLAs

SLI (Service Level Indicator): A quantitative measure of service quality (e.g., latency, error rate).

SLO (Service Level Objective): Target value for an SLI (e.g., P99 latency < 1000ms).

SLA (Service Level Agreement): Contract with consequences for missing SLOs.

Error Budget: Allowed downtime before SLO violation (100% - SLO).

SYNTRIX KPIs: Success Rate ≥99.5%, P50 ≤100ms, P90 ≤300ms, P99 ≤1000ms.

4.19 Artifact Registry & Container Image Management

Managed service for storing Docker images, npm packages, Maven artifacts, etc.

Key Concepts: Regional vs multi-regional repositories, image scanning (vulnerability analysis), image tagging strategies (semver, git SHA, latest), image pull policies in Kubernetes, gcr.io vs Artifact Registry,

authentication (gcloud auth configure-docker), cross-architecture images (linux/amd64 for GCP).

4.20 Python Web Frameworks — FastAPI & Uvicorn

FastAPI: Modern, high-performance Python web framework with automatic OpenAPI docs.

Key Concepts: ASGI (Async Server Gateway Interface), path operations, dependency injection, Pydantic models for validation, middleware (Starlette BaseHTTPMiddleware), async/await, httpx.AsyncClient for non-blocking HTTP calls.

Uvicorn: ASGI server that runs FastAPI. --workers flag for multi-process mode.

SQLAlchemy: ORM for database operations with session management via Depends(get_db).

5. Quick-Revision Cheat Sheet

Key Commands Used in SYNTRIX

Category	Command	Purpose
Docker	<code>docker buildx build --platform linux/amd64 -t <tag> --push .</code>	Cross-build & push image
Docker	<code>docker-compose up --build</code>	Run entire stack locally
Cloud Run	<code>gcloud run deploy <name> --image= --region=<r></code>	Deploy container to Cloud Run
GKE	<code>gcloud container clusters create-auto <name> --region=<r></code>	Create Autopilot cluster
GKE	<code>kubectl create secret generic <name> --from-literal=<k>=<v></code>	Create K8s secret
GKE	<code>kubectl apply -f -</code>	Apply K8s manifest from stdin
GCE	<code>gcloud compute instances create <name> --metadata-from-file=startup-script=mt-with</code>	Update VM with startup script
Cloud SQL	<code>gcloud sql instances create <name> --database-version=POSTGRES_15</code>	Create managed PostgreSQL
Secret Mgr	<code>gcloud secrets create <name> --replication-policy=automatic</code>	Create a secret
IAM	<code>gcloud projects add-iam-policy-binding <proj> --member=<sa> --role=<role></code>	Grant IAM role
Logging	<code>gcloud logging read '<filter>' --format=json --limit=N</code>	Read logs from Cloud Logging
Monitoring	<code>gcloud alpha monitoring policies create --policy-from-file=<f></code>	Create alert policy
BigQuery	<code>bq load --autodetect --source_format=NEWLINE_DELIMITED_JSON <dst> <src></code>	Load NDJSON to BQ
GCS	<code>gsutil cp <local> gs://<bucket>/<path></code>	Upload file to GCS
Registry	<code>gcloud auth configure-docker <region>-docker.pkg.dev</code>	Authenticate Docker to AR

Critical Numbers to Remember

Metric	Value	Context
Services	5 + 1 Gateway	Catalog, Cart, Order, Payment, Inventory + API Gateway
Cloud Run Services	3	Gateway, Catalog, Cart
GKE Services	2	Order, Payment
GCE Services	1	Inventory
BigQuery Views	6	cleaned_logs, latency, error, usage, kpis, cost
Dashboard Sheets	6	Executive, Performance, Errors, Infra, Cost, Log Explorer
Alert Policies	5	Cloud Run 5xx, GKE CPU, GCE CPU, BQ time, Pipeline errors
Region	asia-south1	Mumbai, India
DB Version	PostgreSQL 15	Cloud SQL db-f1-micro
GKE Mode	Autopilot	Fully managed nodes
GCE Machine	e2-micro	0.25 vCPU, 1 GB RAM

Python	3.12-slim	Docker base image
Framework	FastAPI + Uvicorn	ASGI web framework
Success Rate Target	≥ 99.5%	Global SLO
P99 Latency Target	≤ 1000ms	Tail latency SLO

GCP Service Comparison — When to Use What

Criteria	Cloud Run	GKE	Compute Engine
Best For	Stateless HTTP APIs	Complex orchestration	Stateful/legacy workloads
Scaling	Auto (0 to N)	HPA/VPA	Instance Groups + MIG
Pricing	Per-request	Per-pod (Autopilot)	Per-VM-hour
Management	Fully managed	Semi-managed	Self-managed
Cold Start	Yes (1-2s)	No	No
Networking	HTTPS endpoint	K8s Services + ILB	Firewall rules + LB
Secrets	Env vars / SM	K8s Secrets	Instance metadata
Use in SYNTRIX	Gateway, Catalog, Cart	Order, Payment	Inventory

■ **Remember:** You built a real-world cloud-native observability platform using 14+ GCP services. Focus on *why* you made each architectural decision, not just *what* you did. Good luck with the evaluation! — Team SYNTRIX ■